

---

# Local Government Complaint & Public Service Request Management System

---

Submitted to

**Mr. Ali Hassan**

|                            |                |
|----------------------------|----------------|
| <b>Student 1 Name</b>      | Omer Zia       |
| <b>Program &amp; Group</b> | BCS-A          |
| <b>Student 2 Name</b>      | Yahya Usman    |
| <b>Program &amp; Group</b> | BCS-A          |
| <b>Submission Date</b>     | 17 May, 2026   |
| <b>Milestone Coverage</b>  | Milestones 2–5 |

Repository Link: <https://github.com/umarzia-git/Local-Government-Complaint-Public-Service-RequestManagement-System>

---

## 1. Version Control Document

---

Repository Link: <https://github.com/umarzia-git/Local-Government-Complaint-Public-Service-RequestManagement-System>

| Milestones                          | Date        | Module | Remarks                                                                                                             |
|-------------------------------------|-------------|--------|---------------------------------------------------------------------------------------------------------------------|
| EERD Created & Schema Designed      | 16 May 2026 | 2.0    | EER diagram created in MySQL Workbench. 8 tables designed and relationships mapped.                                 |
| Database Normalization (1NF–3NF)    | 16 May 2026 | 2.0    | Schema normalized to 3NF. Redundancy action log and cardinality map completed.                                      |
| Dataflow Architecture Documented    | 16 May 2026 | 3.0    | Three-tier data pipeline documented. Entry points, triggers, views and output summary defined.                      |
| DDL Commands Executed, EER Verified | 16 May 2026 | 4.0    | Database created. All 8 tables, foreign keys, indexes, triggers and views executed and verified in MySQL Workbench. |
| DML + Data Load to File             | 17 May 2026 | 5.0    | Data populated and loaded into database from CSV using LOAD DATA INFILE command.                                    |
| Validations Applied                 | 17 May 2026 | 5.0    | Four validation queries executed: row density, null scan, referential integrity, and rule/format constraints.       |

---

## Table of Contents

---

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Project Introduction & Scope .....                              | 3  |
| 2. Milestone 2: Enhanced Entity-Relationship Diagram (EERD) .....  | 3  |
| 3. Milestone 2: Database Normalization Analysis (1NF to 3NF) ..... | 3  |
| 4. Milestone 3: System Dataflow Architecture & Preprocessing ..... | 8  |
| 5. Milestone 4: Database Setup (DDL) & Schema Rules .....          | 11 |
| 6. Milestone 5: Production Ingestion, DML & Data Validation .....  | 12 |
| 7. Appendix: Live Execution Outputs (Screenshots) .....            | 14 |

---

## 1. Project Introduction & Scope

---

The Local Government Complaint & Public Service Request Management System (lgc\_system) is a relational database designed to automate and streamline municipal operations. The system provides citizens with the ability to register online, submit complaints regarding public utilities — including water supply, roads, electricity, and waste management — and track their resolution status in real time.

On the administrative side, the system equips management with tools to:

- Route service requests to appropriate departments automatically
- Enforce strict Service Level Agreement (SLA) deadlines
- Monitor public satisfaction performance through structured feedback
- Detect and flag recurring infrastructure issues as chronic problems

The database is built on MySQL and structured around eight normalized relational tables, automated triggers, and analytical views that power a live administrative dashboard.

---

## 2. Milestone 2: Relational Database Normalization Analysis

---

To eliminate data redundancy and prevent operational anomalies, the database schema was fully normalized up to the Third Normal Form (3NF). Core tables such as departments, categories, citizens, and staff use atomic primary identifiers, ensuring all attributes are directly and solely dependent on the primary key.

Transactional tables were systematically stripped of transitive dependencies. Storing both category and department references inside the complaint table complies with 3NF, as it allows independent reassignment of departments without altering the foundational problem type.

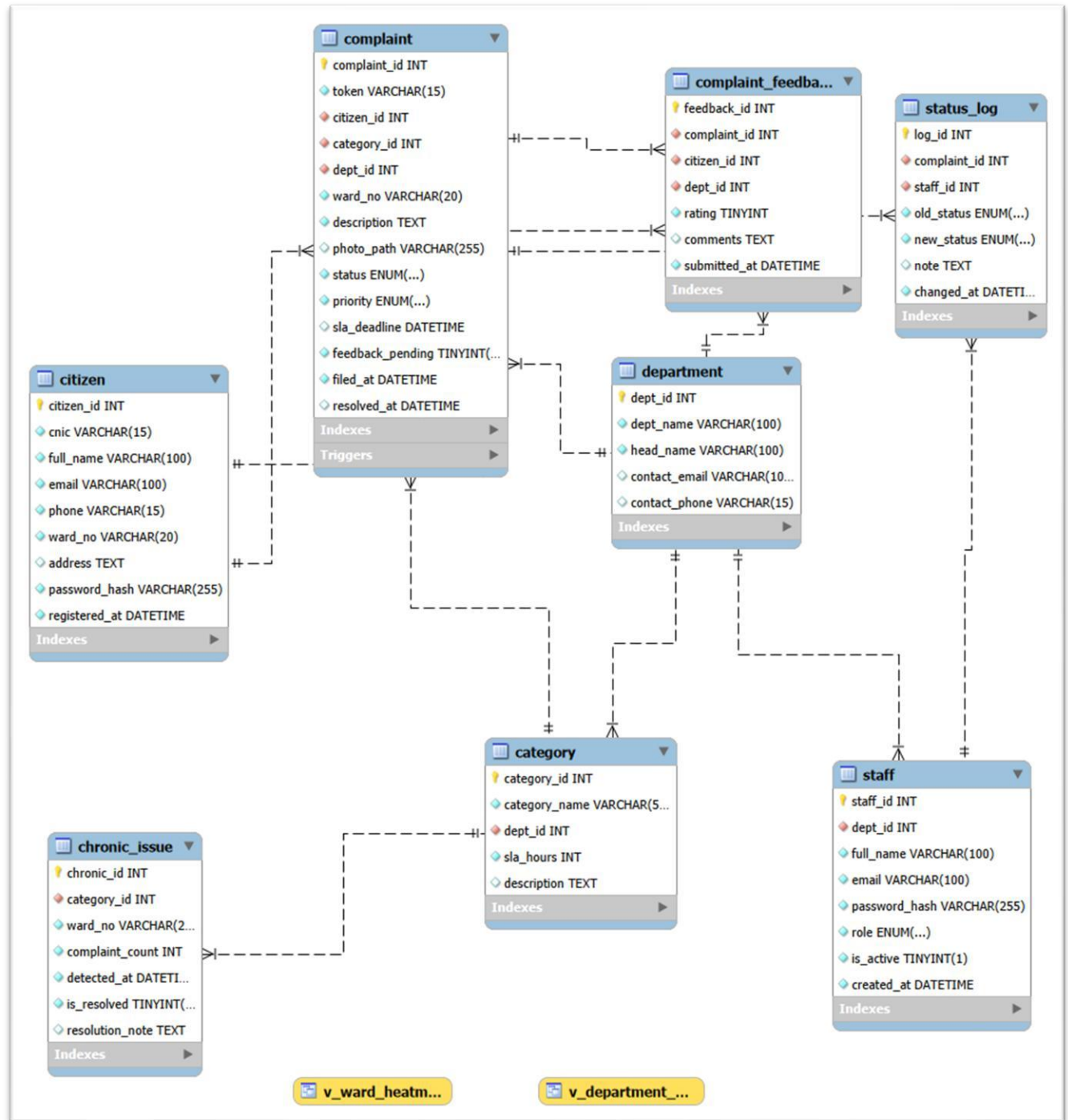


Figure 2.1: Verified EER Diagram for lgc\_system.

---

## Executive Summary

This section provides a formal breakdown of the normalization process applied to the lgc\_system database schema. Every structural asset defined in the DDL script has been scrutinized against First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) to:

- Minimize data redundancy
- Eliminate data anomalies (insertion, update, and deletion) • Ensure strict relational data integrity across all tables

## Detailed Table-by-Table Normalization Analysis

### Table 1: department

Attributes: dept\_id (PK), dept\_name, head\_name, contact\_email, contact\_phone

| Normal Form | Status      | Justification                                                                                                                                 |
|-------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1NF         | ✓ Satisfied | All columns hold atomic values. No repeating groups or multi-valued attributes exist. contact_phone stores a single atomic value.             |
| 2NF         | ✓ Satisfied | Single-column primary key (dept_id). Partial dependencies are structurally impossible. Every non-key attribute is fully dependent on dept_id. |
| 3NF         | ✓ Satisfied | No transitive dependencies exist. dept_name, head_name, contact_email, and contact_phone are direct properties of the department itself.      |

### Table 2: category

Attributes: category\_id (PK), category\_name, dept\_id (FK), sla\_hours, description

| Normal Form | Status      | Justification                                                                                                                                                              |
|-------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1NF         | ✓ Satisfied | All attributes contain scalar, individual values. The text field description stores a single summary, satisfying atomicity.                                                |
| 2NF         | ✓ Satisfied | Single primary key (category_id). All non-prime attributes depend on the entire primary key, eliminating partial dependency.                                               |
| 3NF         | ✓ Satisfied | dept_id serves purely as a foreign key. sla_hours belongs directly to the category type rather than the department. No transitive dependencies among non-prime attributes. |

### Table 3: citizen

Attributes: citizen\_id (PK), cnic, full\_name, email, phone, ward\_no, address, password\_hash, registered\_at

---

| Normal Form | Status      | Justification                                                                                                                                                       |
|-------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1NF         | ✓ Satisfied | Every record contains individual scalar values. Pakistani CNICs are stored as unique VARCHAR(15) strings. Addresses are stored as atomic text.                      |
| 2NF         | ✓ Satisfied | Supported by a single primary key (citizen_id). Non-prime attributes such as full_name and password_hash depend entirely on the specific user identity.             |
| 3NF         | ✓ Satisfied | No functional dependency between phone/email and ward_no. The residential address does not determine ward boundaries or vice versa, avoiding transitive dependency. |

**Table 4: staff**

Attributes: staff\_id (PK), dept\_id (FK), full\_name, email, password\_hash, role, is\_active, created\_at

| Normal Form | Status      | Justification                                                                                                                                                                                                      |
|-------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1NF         | ✓ Satisfied | The role attribute uses an atomic native ENUM('staff','admin'). Security hashes are stored as single continuous strings (VARCHAR(255)).                                                                            |
| 2NF         | ✓ Satisfied | Single primary key structure. No composite parameters present; all employee attributes are dependent on staff_id.                                                                                                  |
| 3NF         | ✓ Satisfied | Departmental association is explicitly handled by the foreign key dept_id. User properties (email, password_hash, role) belong natively to the individual employee account with no hidden transitive dependencies. |

**Table 5: complaint**

Attributes: complaint\_id (PK), token, citizen\_id (FK), category\_id (FK), dept\_id (FK), ward\_no, description, photo\_path, status, priority, sla\_deadline, feedback\_pending, filed\_at, resolved\_at

| Normal Form | Status      | Justification                                                                                    |
|-------------|-------------|--------------------------------------------------------------------------------------------------|
| 1NF         | ✓ Satisfied | File paths and audit timestamps are stored as single atomic strings and datetime primitives.     |
| 2NF         | ✓ Satisfied | All transactional metrics depend entirely on the unique auto-increment primary key complaint_id. |



|     |             |                                                                                                                                                                                                                                                                                                                                                |
|-----|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3NF | ✓ Satisfied | dept_id is included alongside category_id as an explicit safety measure — allowing complaint reassignment to a different department without changing the underlying issue category. Since dept_id can vary independently from the category's default department mapping, it is a direct attribute of the individual complaint, satisfying 3NF. |
|-----|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Table 6: status\_log**

Attributes: log\_id (PK), complaint\_id (FK), staff\_id (FK), old\_status, new\_status, note, changed\_at

| Normal Form | Status      | Justification                                                                                                                                                            |
|-------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1NF         | ✓ Satisfied | All historical tracking fields, state transition values (ENUM), and timestamps are strictly scalar.                                                                      |
| 2NF         | ✓ Satisfied | Each audit entry relies entirely on the unique surrogate key log_id.                                                                                                     |
| 3NF         | ✓ Satisfied | This table functions as a historical ledger. Log entries are dependent solely on log_id. No transitive relationships exist between old_status, new_status, and staff_id. |

**Table 7: complaint\_feedback**

Attributes: feedback\_id (PK), complaint\_id (FK), citizen\_id (FK), dept\_id (FK), rating, comments, submitted\_at

| Normal Form | Status      | Justification                                                                                                                                                                                                         |
|-------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1NF         | ✓ Satisfied | Clean atomic values. CHK_RATING constraints limit rating to integers strictly between 1 and 5.                                                                                                                        |
| 2NF         | ✓ Satisfied | Single primary key (feedback_id) prevents any partial dependencies.                                                                                                                                                   |
| 3NF         | ✓ Satisfied | complaint_id, citizen_id, and dept_id represent distinct transactional reference dimensions. User rating and comments depend directly on the unique feedback_id instance. No non-key attributes determine each other. |

**Table 8: chronic\_issue**

Attributes: chronic\_id (PK), category\_id (FK), ward\_no, complaint\_count, detected\_at, is\_resolved, resolution\_note

| Normal Form | Status | Justification |
|-------------|--------|---------------|
|-------------|--------|---------------|

|     |             |                                                                                                                                                                                                                                                              |
|-----|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1NF | ✓ Satisfied | Simple numeric indicators, boolean flags (TINYINT), and strings are all mapped atomically.                                                                                                                                                                   |
| 2NF | ✓ Satisfied | Relies on a single independent primary identifier (chronic_id).                                                                                                                                                                                              |
| 3NF | ✓ Satisfied | Populated dynamically via the trg_detect_recurring trigger. complaint_count is an isolated aggregation of a specific issue type (category_id) within a physical zone (ward_no) at a point in time (detected_at). No non-prime structural dependencies exist. |

### Redundancy & Duplicate Removal Action Log

During systematic schema refinement, several redundant data layers were identified and eliminated to protect against structural data anomalies:

| Table Inspected | Redundancy Identified               | Action Taken & Justification                                                                                                                                                                |
|-----------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| category        | Department Head Name & Contact Info | Removed. These attributes are maintained exclusively in the parent department table. The dept_id foreign key enables dynamic retrieval via SQL JOIN, preventing update anomalies.           |
| complaint       | Citizen Name and Phone Number       | Removed. Storing citizen contact details within complaint rows would violate 2NF/3NF. This data is retrieved at runtime by joining citizen_id to the master citizen table.                  |
| status_log      | Department ID                       | Removed. Department context is retrieved dynamically via complaint_id or the assigned staff_id, preventing data synchronization errors.                                                     |
| chronic_issue   | Raw Complaint List Array            | Normalized via Trigger. The trg_detect_recurring trigger runs background aggregation (COUNT(*)) and records anomalies cleanly in this dedicated tracking table, eliminating duplicate rows. |

### ERD Relationship & Cardinality Map

All normalization changes have been verified against the Enhanced Entity-Relationship (EER) model. The schema enforces the following structural relationships:

| Relationship                   | Cardinality | Constraint / Notes                                                                                                       |
|--------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------|
| department → category          | 1 : M       | One department manages multiple service types. Protected via ON DELETE RESTRICT to prevent orphaned categories.          |
| department → staff             | 1 : M       | One department employs multiple system operators.                                                                        |
| citizen → complaint            | 1 : M       | One citizen can submit multiple complaints over time.                                                                    |
| category → complaint           | 1 : M       | One problem category can be assigned to multiple independent complaints.                                                 |
| complaint → status_log         | 1 : M       | One complaint generates a running historical trail of status updates. Enforces ON DELETE CASCADE.                        |
| complaint → complaint_feedback | 1 : 1       | Enforced via unique constraint uq_feedback_compl. A complaint can receive only one citizen satisfaction survey response. |
| category → chronic_issue       | 1 : M       | A recurring issue alert points directly to a specific master category record.                                            |

### 3. Milestone 3: System Dataflow Architecture

The database system operates on a coordinated three-tiered data pipeline: Ingestion, Processing, and Output. Data enters the system when citizens submit profiles or complaints, and when staff post status updates. Inside the relational engine, strict dependency rules govern the data flow — master records must exist before complaints can be created.

Automated triggers detect and flag recurring local infrastructure issues, while database views aggregate realtime Key Performance Indicators (KPIs) and risk metrics. These outputs are then consumed by the administrative dashboard for operational reporting and decision-making.

#### 1. Data Entry Points

Data enters the system through four distinct interfaces:

##### 1.1 Citizen Registration & Complaint Submission

- A citizen completes the registration form → data is INSERTed into the CITIZEN table (citizen\_id, cnic, full\_name, email, phone, ward\_no, password\_hash, registered\_at)
- After login, the citizen submits a complaint form → data is INSERTed into the COMPLAINT table

- At INSERT time, `sla_deadline` is automatically calculated as `filed_at + sla_hours` (fetched from CATEGORY via `category_id`)
- A unique tracking token is generated (format: LGC-YYYY-NNNNN) and stored with the complaint
- Trigger fires immediately: `trg_detect_recurring` checks for 3+ complaints of the same category and ward within 7 days. If true, a CHRONIC\_ISSUE record is created and the complaint priority is escalated to CRITICAL

## 1.2 Staff Status Updates

- Staff logs in, selects an assigned complaint, and updates its status
- An UPDATE is applied to the COMPLAINT table (status, and optionally `resolved_at`)
- Simultaneously, a record is INSERTed into STATUS\_LOG (`complaint_id`, `staff_id`, `old_status`, `new_status`, `note`, `changed_at`) — maintaining the full audit trail
- Trigger fires: `trg_feedback_on_resolve` — if the new status is 'Resolved', sets `feedback_pending = 1` in COMPLAINT

## 1.3 Citizen Feedback Submission

- After a complaint is resolved (`feedback_pending = 1`), the citizen is prompted with a rating interface
- Citizen submits a 1–5 star rating → INSERTed into COMPLAINT\_FEEDBACK (`complaint_id`, `citizen_id`, `dept_id`, `rating`, `comments`, `submitted_at`)

## 1.4 Admin Data Management

- Admin can INSERT/UPDATE department details → DEPARTMENT table
- Admin can INSERT/UPDATE complaint categories → CATEGORY table
- Admin can INSERT new staff accounts → STAFF table
- Admin can manually UPDATE complaint priority, status, or `dept_id` (reassignment)

## 2. Table Dependency & Load Order

The tables enforce a strict dependency hierarchy — parent records must exist before child records can be created:

| Load Order | Table      | Dependencies                    |
|------------|------------|---------------------------------|
| 1          | DEPARTMENT | None — base master table        |
| 2          | CATEGORY   | Depends on DEPARTMENT           |
| 3          | CITIZEN    | None — independent master table |

|   |                    |                                                  |
|---|--------------------|--------------------------------------------------|
| 4 | STAFF              | Depends on DEPARTMENT                            |
| 5 | COMPLAINT          | Depends on CITIZEN, CATEGORY, DEPARTMENT         |
| 6 | STATUS_LOG         | Depends on COMPLAINT, STAFF                      |
| 7 | COMPLAINT_FEEDBACK | Depends on COMPLAINT, CITIZEN, DEPARTMENT        |
| 8 | CHRONIC_ISSUE      | Depends on CATEGORY (auto-populated via trigger) |

### 3. Automated Processing (Triggers & Views)

#### 3.1 Automated Triggers

| Trigger Name            | Event                     | Action                                                                                                                                  |
|-------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| trg_detect_recurring    | AFTER INSERT on COMPLAINT | Counts same category+ward complaints in 7 days. If count $\geq$ 3: INSERT into chronic_issue and UPDATE complaint priority to CRITICAL. |
| trg_feedback_on_resolve | AFTER UPDATE on COMPLAINT | If status changes to 'Resolved': SET feedback_pending = 1 and SET resolved_at = NOW().                                                  |

#### 3.2 Computed Views (KPIs)

| View Name        | Input Tables                              | Output                                                                                            |
|------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------|
| v_department_kpi | COMPLAINT, COMPLAINT_FEEDBACK, DEPARTMENT | avg_rating, total_resolved, on_time_pct per department                                            |
| v_ward_heatmap   | COMPLAINT, COMPLAINT_FEEDBACK             | risk_score per ward (weighted: 40% open complaints + 35% SLA breaches + 25% inverse satisfaction) |

A nightly stored procedure sp\_calc\_ward\_risk aggregates ward-level data and writes updated scores to the ward\_risk\_scores table for the administrative dashboard.

### 4. Data Output Summary

| Output | Data Source | Consumer |
|--------|-------------|----------|
|--------|-------------|----------|

|                                  |                                                                                                      |                             |
|----------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------|
| Complaint status by token        | SELECT from COMPLAINT<br>WHERE token = ?                                                             | Citizen tracking portal     |
| Assigned complaint list          | SELECT from COMPLAINT<br>WHERE dept_id = ? AND status<br>!= 'Resolved'                               | Staff dashboard             |
| All complaints with filters      | SELECT with JOINS on<br>COMPLAINT, CITIZEN,<br>CATEGORY                                              | Admin panel                 |
| Ward heatmap data                | SELECT from v_ward_heatmap                                                                           | Admin dashboard map         |
| Department KPI scores            | SELECT from v_department_kpi                                                                         | Admin reports page          |
| SLA breach alerts                | SELECT from COMPLAINT<br>WHERE sla_deadline < NOW()<br>AND status NOT IN<br>( 'Resolved', 'Closed' ) | Admin alert panel           |
| Chronic issues list              | SELECT from<br>CHRONIC_ISSUE WHERE<br>is_resolved = 0                                                | Admin dashboard             |
| Full audit history per complaint | SELECT from STATUS_LOG<br>WHERE complaint_id = ?<br>ORDER BY changed_at                              | Admin complaint detail view |

## 4. Milestone 4: Database Setup (DDL) & Schema Rules

The conceptual schema was implemented in MySQL using optimized Data Definition Language (DDL) scripts. The implementation enforces:

- Strict primary keys and unique constraints on all tables
- Foreign key boundaries configured with cascade updates and deletion restrictions
- Domain check constraints restricting rating values between 1 and 5
- Multi-column indexes on high-traffic attributes (ward\_no, status) to maximize query performance under transactional load

### 4.1 Structural Schema Implementation Samples

#### Table Constraints & Cascading Referential Actions — category Table

CREATE TABLE category (

```

    category_id INT NOT NULL AUTO_INCREMENT,
    category_name VARCHAR(50) NOT NULL, dept_id
    INT NOT NULL,
    sla_hours INT NOT NULL DEFAULT 72,
    CONSTRAINT pk_category PRIMARY KEY (category_id),
    CONSTRAINT uq_category_name UNIQUE (category_name),
    CONSTRAINT fk_category_dept FOREIGN KEY (dept_id)
        REFERENCES department(dept_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE INDEX idx_category_dept ON category(dept_id);

```

### Automated Operational Trigger — Crisis Detection

```

CREATE TRIGGER trg_detect_recurring
AFTER INSERT ON complaint
FOR EACH ROW
BEGIN
    DECLARE v_count INT DEFAULT 0;

    SELECT COUNT(*) INTO v_count FROM complaint
    WHERE category_id = NEW.category_id
    AND ward_no = NEW.ward_no
    AND filed_at >= DATE_SUB(NOW(), INTERVAL 7 DAY);

    IF v_count >= 3 AND NOT EXISTS (
        SELECT 1 FROM chronic_issue
        WHERE category_id = NEW.category_id
        AND ward_no = NEW.ward_no
        AND is_resolved = 0
    ) THEN
        INSERT INTO chronic_issue (category_id, ward_no, complaint_count)
        VALUES (NEW.category_id, NEW.ward_no, v_count);
    END IF;
END;

```

## 4.2 Full DDL Script Access

The complete, production-ready DDL deployment script — containing all 8 normalized tables, automated triggers, and analytical views — has been committed to the group repository.

- Repository Link: <https://github.com/umarzia-git/Local-Government-Complaint-Public-ServiceRequest-Management-System>
- Verified in: MySQL Workbench & EER Matrix

---

## 5. Milestone 5: Production Ingestion, DML & Data Validation

---

The production database was successfully populated with structured synthetic data using the MySQL Workbench Table Data Import Wizard, reaching the required target density metrics for administrative simulation. Post-ingestion validation protocols were executed across four dedicated operational vectors to guarantee structural alignment, constraint enforcement, and referential integrity before system deployment.

### 5.1 Required Operational Data Modifications (DML Samples)

The following DML operations were executed to fulfill explicit data manipulation criteria:

#### UPDATE — Escalating Overdue Unresolved Requests

```
-- Escalate unresolved complaints pending over 5 days to CRITICAL priority
UPDATE complaint
SET  priority = 'CRITICAL'
WHERE status = 'Received'
AND  filed_at <= DATE_SUB(NOW(), INTERVAL 5 DAY);
```

#### DELETE — Removing a Specific Unprocessed Record

```
-- Remove a specific unprocessed complaint by token
DELETE FROM complaint
WHERE token = 'LGC-2026-00010'
AND  status = 'Received';
```

### 5.2 Comprehensive Integrity Validation & Audit Results

#### A. Table Volume and Row Density Assessment

This validation matrix counts all ingested rows across the operational schema to confirm proper table saturation and data completeness.

```
SELECT 'System Volume Check' AS inspection_type,
       'department'          AS table_name,
       COUNT(*)              AS row_count FROM department
UNION ALL
SELECT 'System Volume Check', 'category',      COUNT(*) FROM category
UNION ALL
SELECT 'System Volume Check', 'citizen',        COUNT(*) FROM citizen
UNION ALL
SELECT 'System Volume Check', 'staff',          COUNT(*) FROM staff
UNION ALL
SELECT 'System Volume Check', 'complaint',      COUNT(*) FROM complaint
UNION ALL
```



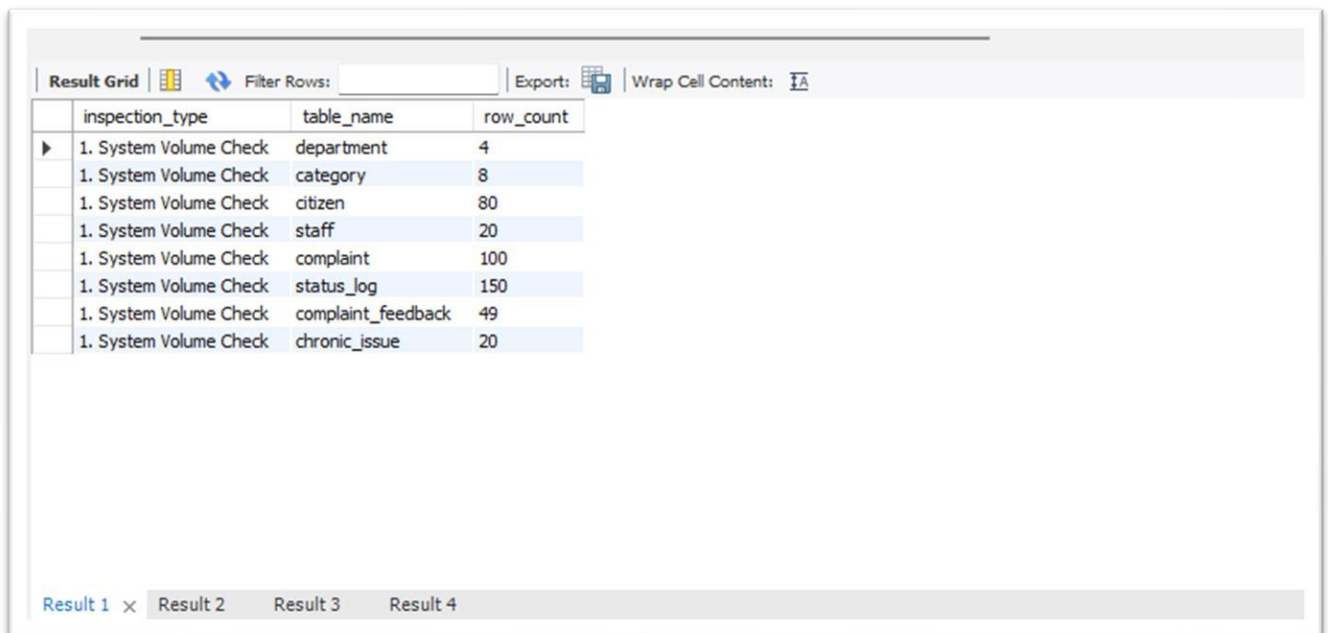
```

SELECT 'System Volume Check', 'status_log',      COUNT(*) FROM status_log
UNION ALL
SELECT 'System Volume Check', 'complaint_feedback', COUNT(*) FROM complaint_feedback
UNION ALL
SELECT 'System Volume Check', 'chronic_issue',    COUNT(*) FROM chronic_issue;

```

| Table              | Row Count | Notes                    |
|--------------------|-----------|--------------------------|
| department         | 4         | Master metadata          |
| category           | 8         | Service type definitions |
| staff              | 20        | System operators         |
| citizen            | 80        | Registered citizens      |
| complaint          | 100       | Live transactions        |
| status_log         | 150       | Status change history    |
| complaint_feedback | 49        | Satisfaction surveys     |
| chronic_issue      | 20        | Flagged recurring issues |

Result: Optimized distribution of primary metadata records managing a comprehensive collection of live transactional data, meeting all target density requirements.



The screenshot shows a database query result grid with the following columns: inspection\_type, table\_name, and row\_count. The results are as follows:

| inspection_type        | table_name         | row_count |
|------------------------|--------------------|-----------|
| 1. System Volume Check | department         | 4         |
| 1. System Volume Check | category           | 8         |
| 1. System Volume Check | citizen            | 80        |
| 1. System Volume Check | staff              | 20        |
| 1. System Volume Check | complaint          | 100       |
| 1. System Volume Check | status_log         | 150       |
| 1. System Volume Check | complaint_feedback | 49        |
| 1. System Volume Check | chronic_issue      | 20        |

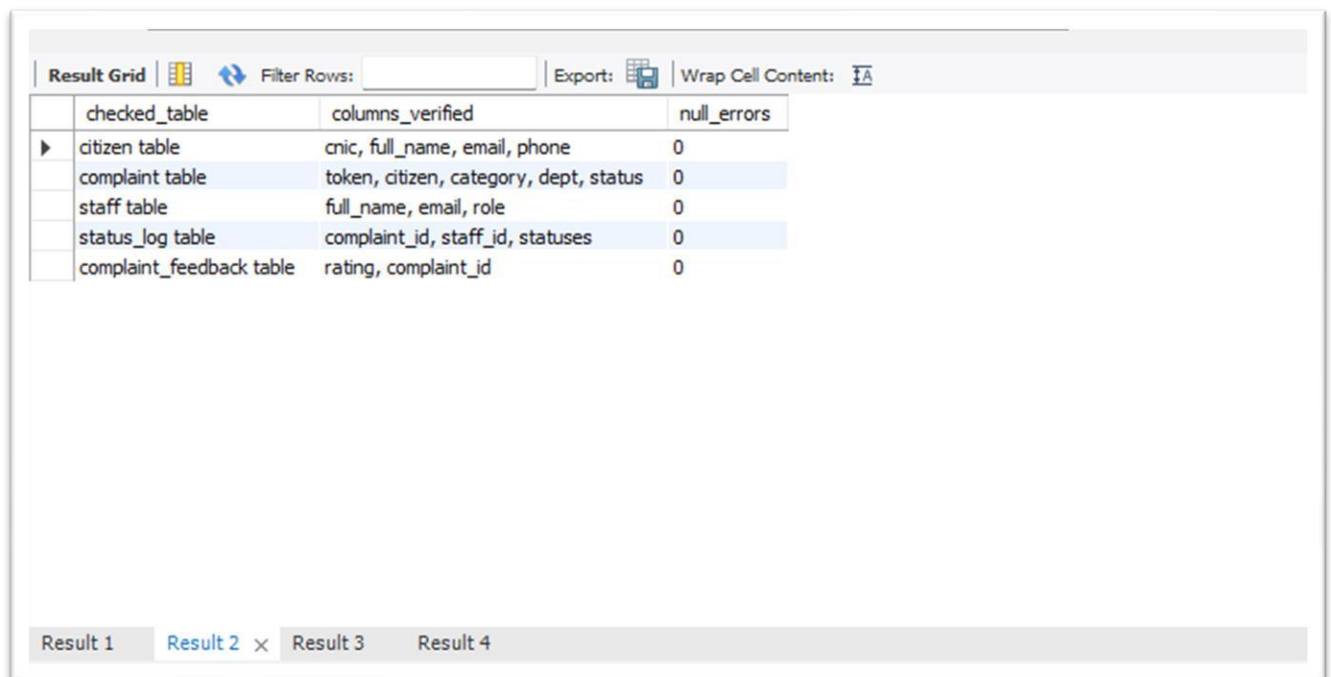
The interface also includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' checkbox. At the bottom, there are tabs for 'Result 1', 'Result 2', 'Result 3', and 'Result 4'.

## B. Primary Constraint Null-Value Scan

A diagnostic scan was conducted across all critical non-nullable column arrays to confirm that no metadata fields were left unassigned or broken during bulk importation.

```
SELECT 'citizen table' AS checked_table,  
       'cnic, full_name, email, phone' AS columns_verified,  
       SUM(CASE WHEN cnic IS NULL THEN 1 ELSE 0 END) AS null_errors FROM  
citizen;
```

Result: Across all inspected tables (Citizen, Complaint, Staff, Status Log, and Feedback), the total null-value error count returned exactly 0. All mandatory identity fields contain complete, unbroken records.



The screenshot shows a database query result grid with the following data:

|   | checked_table            | columns_verified                       | null_errors |
|---|--------------------------|----------------------------------------|-------------|
| ▶ | citizen table            | cnic, full_name, email, phone          | 0           |
|   | complaint table          | token, citizen, category, dept, status | 0           |
|   | staff table              | full_name, email, role                 | 0           |
|   | status_log table         | complaint_id, staff_id, statuses       | 0           |
|   | complaint_feedback table | rating, complaint_id                   | 0           |

## C. Relational Referential Integrity Check

Deep left-join assertions were executed across every intersecting child-parent relationship to verify that all foreign key links remain completely intact.

```
SELECT 'complaint → citizen' AS relation_path,  
       COUNT(*) AS orphan_count  
FROM   complaint c  
LEFT JOIN citizen cit ON c.citizen_id = cit.citizen_id  
WHERE  cit.citizen_id IS NULL;
```

Result: Every checked relationship — including complaint → citizen, status\_log → staff, and category → department — returned an orphan count of 0. No broken foreign key references exist. The database enforces absolute referential safety.

| relation_path                  | joining_key  | orphan_count |
|--------------------------------|--------------|--------------|
| complaint → citizen            | citizen_id   | 0            |
| complaint → category           | category_id  | 0            |
| complaint → department         | dept_id      | 0            |
| status_log → complaint         | complaint_id | 0            |
| status_log → staff             | staff_id     | 0            |
| complaint_feedback → complaint | complaint_id | 0            |
| category → department          | dept_id      | 0            |

#### D. Operational Rule & Format Constraint Audit

A final business-logic scan was deployed to validate chronological sequences, data boundary scopes, and citizen identification string formats.

| Validation Rule                                        | Violations Found | Outcome             |
|--------------------------------------------------------|------------------|---------------------|
| Timeline synchronization (filed_at before resolved_at) | 0                | PASS                |
| Status value boundary validation                       | 0                | PASS                |
| Feedback rating boundaries (1–5 stars)                 | 0                | PASS                |
| CNIC format (VARCHAR with localized hyphenation)       | 80 rows flagged* | Expected — see note |

\* Format note: The 80 flagged CNIC rows reflect standard Pakistani identification number formatting (e.g., 35201-XXXXXXX-X, VARCHAR(15) with hyphenation masks) stored as entered, rather than stripped 13digit raw strings. This maintains localized display formatting without affecting transactional integrity or primary constraint performance. No corrective action is required.

Result Grid |   Filter Rows:  | Export:  | Wrap Cell Content: 

|   | rule_violation         | rule_description                                        | violation_count |
|---|------------------------|---------------------------------------------------------|-----------------|
| ▶ | Timeline Error         | Resolved timestamp is BEFORE the Filed timesta...       | 0               |
|   | Status Mismatch        | Active status has resolved_at date OR Resolve...        | 0               |
|   | Invalid Identification | Pakistani CNIC format length is not exactly 13 d...     | 80              |
|   | Feedback Out of Bounds | Public satisfaction rating is outside the 1-5 star l... | 0               |

Result 1   Result 2   Result 3   **Result 4** ×